INDEX

NAME: MOHAMED ISMAIL FAWAZ K P ROLL NO.: 240701324

STD.: _____ DIV./SEC.: _____ SUBJECT: COMPUTER NETWORKS

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
1	21/07/26	Study of Various Network Commands used in Linux & Command		
2	27/07/26	Study of Network Cables		
3	28/07/26	Study of Packet Tracer Tool Installation & User Interface Overview		
4	03/08/26	Setting up a Local Area Network		
5	04/08/26	Understanding & Studying the Wireshark Tool		
6	10/08/26	Error Detection & Correction Using Hamming code		
7	11/08/26	Sliding Window Protocol		
8	18/08/26	a) Nmap Scans ON TRYHACKME		
9	24/08/26	b) Configuration of Wireless LAN Using Cisco Packet Tracer		
9	25/08/26	Implementation of Subnetting in Cisco packet Tracer.		
10	31/08/26	A) Internetworking with Routers In Cisco Packet Tracer Simulator		
	08/09/26	B) Internetworking using A Wireless router, DHCP server & Internet Cloud		

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
11	15/09/26	A) Simulate Static Routing using Cisco Packet Tracer		
	22/09/26	B) Simulate RIP using Cisco Packet Tracer		
12	28/09/26	A) Echo Client-Server Using TCP/UDP sockets		
	29/09/26	B) Chat Client-Server Using TCP/UDP		
13	06/10/26	Implement A Ping Program		
14.	12/12/26	Packet Sniffing using RAW Sockets		
15.	12/12/26	Analysis Of Different Types of Web logs using WebAlizer.		

Exp: 1

STUDY OF VARIOUS NETWORK COMMANDS USED IN LINUX & WINDOWS

AIM:

To study various network commands used in Linux & Windows.

BASIC NETWORKING COMMANDS:

arp-a : Address Resolution Protocol. Shows the IP address of the computer along with

hostname : Displays the name of your computer

ipconfig/all : Displays detailed configuration information about your TCP/IP connection including ROUTER, Gateway, DNS, DHCP

nbstat -a : Solve problems with NetBIOS name resolution.

netstat : commandline tool for monitoring network connections both incoming/outgoing as well as viewing routing tables.

nslookup : perform DNS lookups in linux.

pathping : Unique to windows & is a combination of Ping & Tracert commands.

ping : uses ICMP to communicate to other devices.

#ping hostname

#ping ip address

#ping domain name

route : show / manipulate the IP routing table.

LINUX NETWORKING COMMANDS :

ip :

ip <options> <object> <command>

(a) ip address show : Show ip address assigned to an interface on your server

(b) To assign an IP to an interface :
ip address add 192.168.1.254/24 dev enps03

(c) ip address del 192.168.1.254/24 dev enps 03 :
To delete an IP on an interface.

(d) ip link set eth0 : alter the status of the interface.

(e) ip route add 192.168.1.0/24 via 192.168.1.254
Add a route via the gateway.

ifconfig:

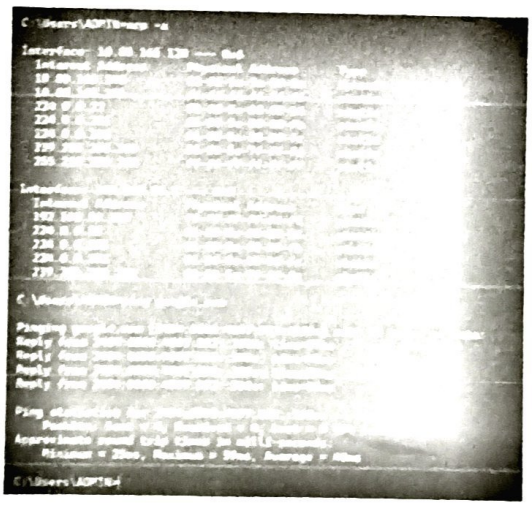
configure & troubleshooting networks.

mtr:

Matt's traceroute. command-line Interface that serves as a network diagnostic & troubleshooting tool

tcpdump:

capture & display packets.



RESULT:

Thus the networking commands are studied & the output is verified.

Exp: 2 STUDY OF NETWORK CABLES

AIM:

To understand the types of cables used in networking such as

(i) Unshielded Twisted Pair (UTP) cable.

(ii) Shielded Twisted Pair (UTP) Cable

(iii) Coaxial Cable

(iv) Fibre Optic Cable.

Cable Type	Category	Maximum Data Transmission	Advantages / Disadvantages	Application / Uses
UTP	Category 3	10 bps	Advantages: Cheap, Easy to Install as they have smaller diameter Disadvantages: More prone to EMI	10 Base-T Ethernet
	Category 5	Up to 100Mbps		Fast Ethernet Gigabit Ethernet
	Category 5e	1 Gbps		
STP	Category 6, 6a	10Gbps	Advantages Shielded, Faster than UTP, less susceptible to noise & Interference Disadvantages: Expensive Greater Installation Effort	Gigabit Ethernet 10G Ethernet Widely used in data centres
SSTP	Category 7	10Gbps		Gigabit Ethernet, 10G Ethernet (100m).

Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages: High bandwidth, Immune to Interference Low loss bandwidth versatile Disadvantages: Limited Distance, Cost, Size is bulky	Speed of signal is Room Television network High speed Internet connection
Fibre Optic Cable	Single Mode & Multi Mode	100Gbps	Advantages: High speed, Bandwidth security, Long Distance Disadvantages: Expensive Requires Skilled Installers	Maximum distance of fibre Optics cable is around 100 meters

Outputs:

Hence the different types of cables used in networking are studied.

Exp: 3 STUDY OF PACKET TRACER TOOL

INSTALLATION & USER INTERFACE OVERVIEW

AIM:

To study & understand cisco packet tracer tool.

Introduction:

Cisco Packet Tracer is a network simulation tool used to design, configure & troubleshoot computer networks. It helps students understand networking concepts, through virtual devices & simulated network environments.

Installing Cisco Packet Tracer:

Cisco Packet Tracer is downloaded from the official cisco Networking academy, website & is installed on the computer by following installation instructions. After installations, the application is launched successfully.

User Interface Overview:

The UI of Cisco Packet Trace is explored. Main components includes Toolbar, workspace, device selection panel, connection tools, simulation controls. These features help users create configure & test network topologies.

Analyse the Behaviour of Network Devices using Cisco Packet Tracer:

* From the network component box, click drag & drop the below components.

a) 4 Generic PC's & one Hub

b) 4 Generic PC's & one switch.

* Click on connections.

* Click on copper straight-through cable

* Select one of the PC & connect it to Hub using cable. The Link LED should glow in green. Similarly connect 4 PC's to the switch

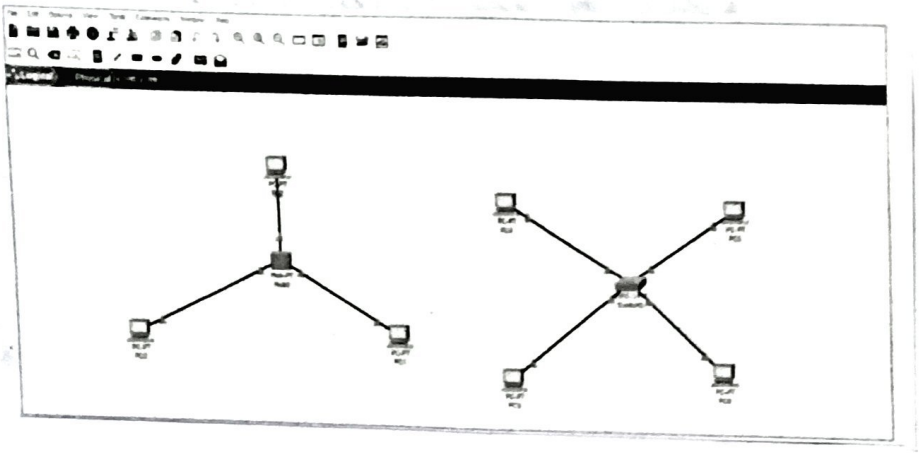
* Click on PC's connected to Hub, go to Desktop tab. Click on IP configuration & enter an IP address & subnet mask.

* Click on the PDU from the common tool bar. Drag & Drop it on one of PC (source machine) & then drop it on another PC.

* Observe the flow of PDU from source PC to Destination PC by selecting the real time mode of simulation.

* Repeat step #3 to step #5 for the PC's connected to the switches.

* Observe how HUB & switch are forwarding the PDU & write your observation



Output:

Hence Cisco Packet Tracer tool is studied & its working is understood.

Exp: 4 SETTING UP A LOCAL AREA NETWORK

AIM:

To design & set up & configure a Local Area Network (LAN) connecting 4 computers through a network switch, assign static IP addresses to all devices.

Equipment Required:

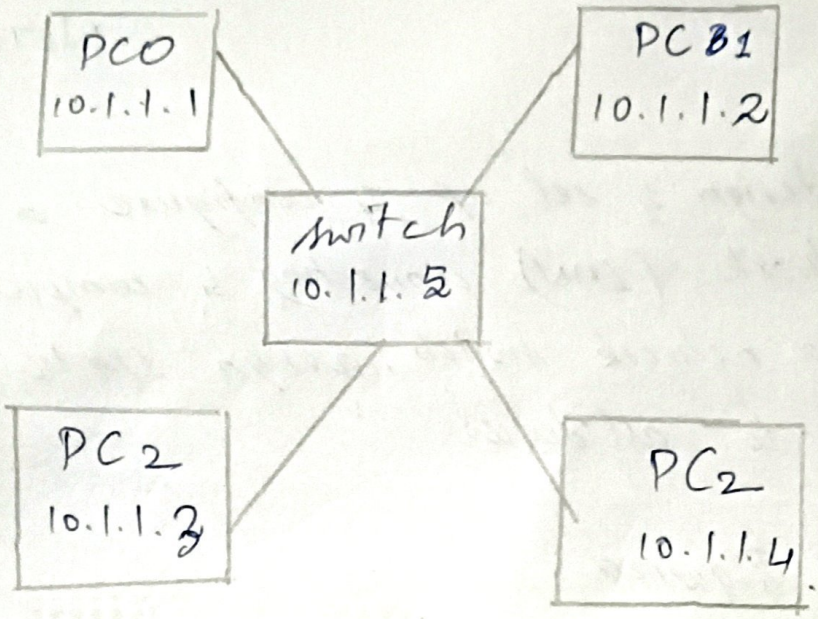
S.No	Equipment	Quantity	Specifications
1	Personal Computer	4	Ethernet (NIC) enabled
2	Network Switch	1	8/16/24 port unmanaged/managed
3	Ethernet Cables (Cat5e/Cat6)	4	Straight-through RJ-45 connector

Theory:

A Local Area Network (LAN) is a network that connects computers & devices within a limited geographical area such as home, office.

A Star Topology is used in this experiment, where all end devices (PC's) are individually connected to a central network switch.

Network Diagram:



IP address configuration Table:

Device	IP address	Subnet Mask	Connected To
PC0	10.1.1.1	255.0.0.0	Switch port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch	10.1.1.5	255.0.0.0	VLAN 1 Interface

Implementation:

* Open the Cisco packet Tracer & place the devices.

* Click connections, select copper straight through cable & cable the devices.

* Click PC → Desktop Tab → IP configuration & assign IP addresses.

* Click switch → CLI Tab & enter the following

```
enable
configure terminal
interface vlan 1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit
```

* Verify Connectivity using ping command

* Add a server device (End device → Server PT) Assign it an IP address via its Desktop → IP configuration

* Open Server's services & enable HTTP service.

* Switch to simulation mode & send a ping or web request between 2 devices.

Result:

All 4 PC's were successfully connected to switch in a star topology & assigned unique static IP addresses within the same subnet.

Exp: 6 UNDERSTANDING & STUDYING THE WIRESHARK TOOL

AIM:

To understand & study the wireshark tool & analyse network packets to observe network communication.

THEORY:

Wireshark is a free, open-source network protocol analyzer used to capture & inspect network packets. It helps understand how data travels across a network & identify different protocols such as TCP, UDP, DNS, HTTP & ICMP

Installations:

Wireshark can be downloaded from its official website https://wireshark.org/. After downloading, run the installer & follow the on-screen instructions to complete the installation Install the required packet, capture component, such as Npcap on windows, to capture network traffic.

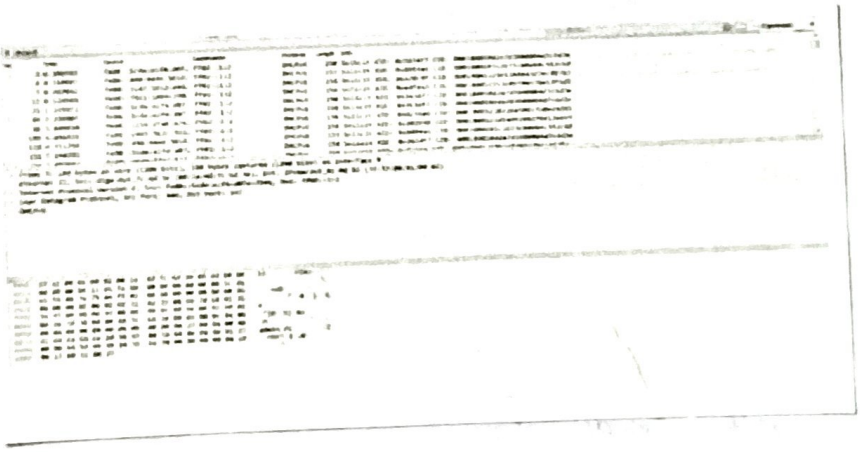
User Interface:

The Wireshark user interface consists of three main panes: the Packet List Pane, the Packet Details Pane & the Packet Bytes Pane. The Packet List Pane displays captured

packets, the packet details pane shows the protocol information of a selected packet, and the Packet Bytes pane displays the packet data in Hexadecimal & Ascii formats.

Capturing & Analysing Packets using Wireshark:

* Select Local area connection in Wireshark
* Go to capture option
* selection stop capture automatically after 100 packets
* Then click start Capture
* Save the packets



1. Create a filter to display only TCP/UDP packets Inspect the packets & provide flow Graph

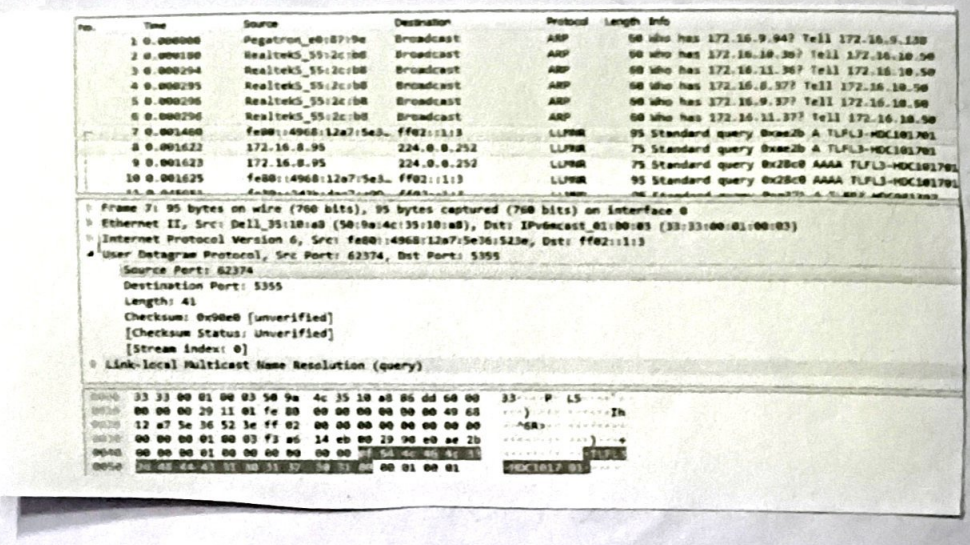
Procedure:

* select local area connection in Wireshark
* Go to capture option
* select stop capture automatically after 100 packets
* Click start capture

* Search TCP packets in search bar

* To see flow graph, click statistics. Flow Graph

* Save the packets.



2. Create filter to display only ARP packets.

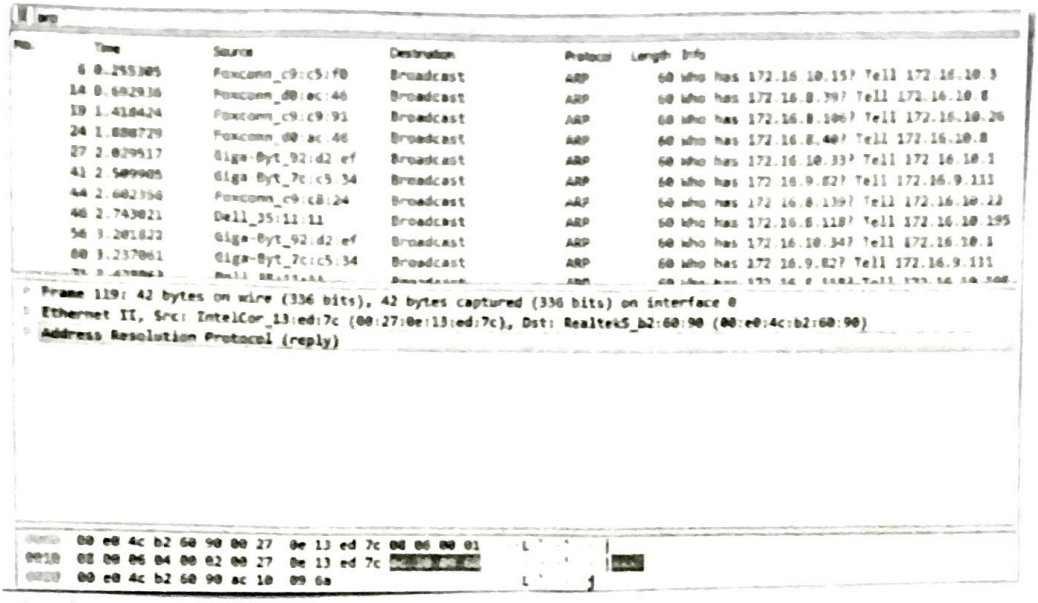
* Go to capture option

* Select start capture automatically after 100 packets

* Then click start capture

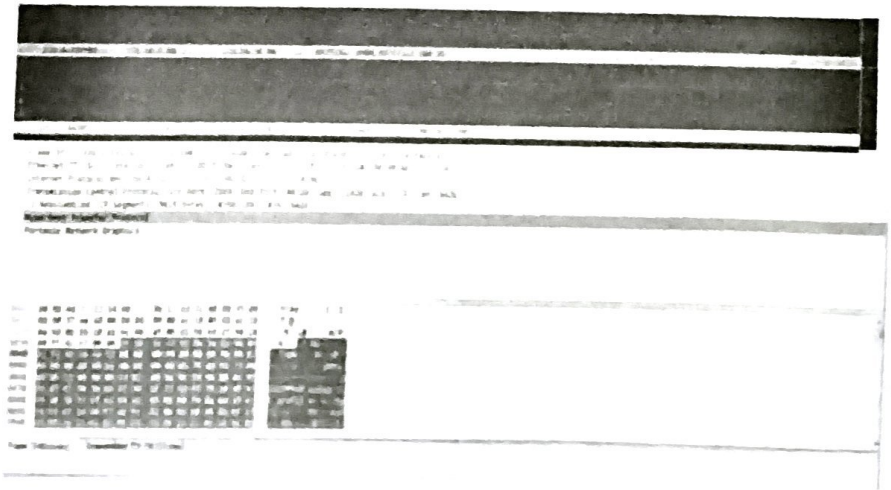
* Search ARP packets in search bar

* Save the bar.



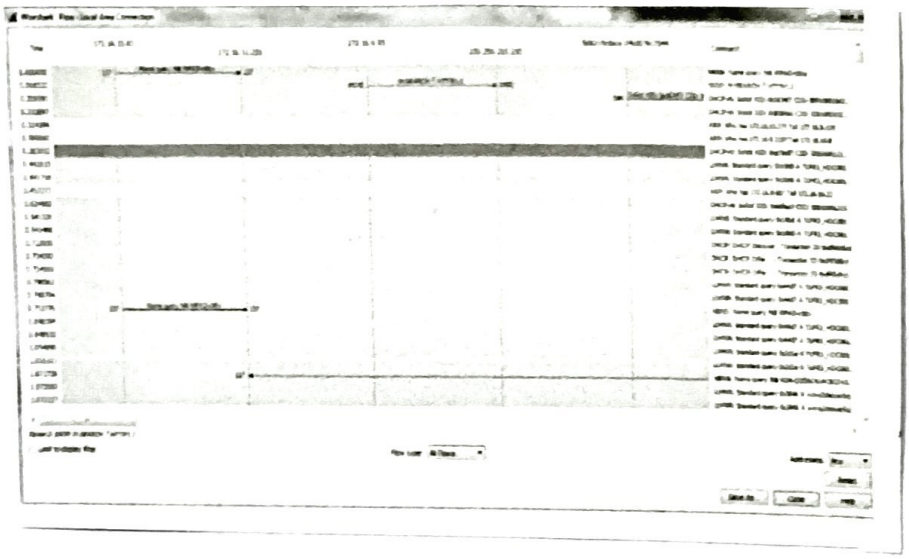
3. Create a filter to display only DNS packets & provide the flow graph.

* Go to ~~procedure~~ capture option
* Select stop capture automatically after 100 packets
* Then click start capture
* Search DNS packets in search bar.
* To see flow graph, click Statistics flow graph



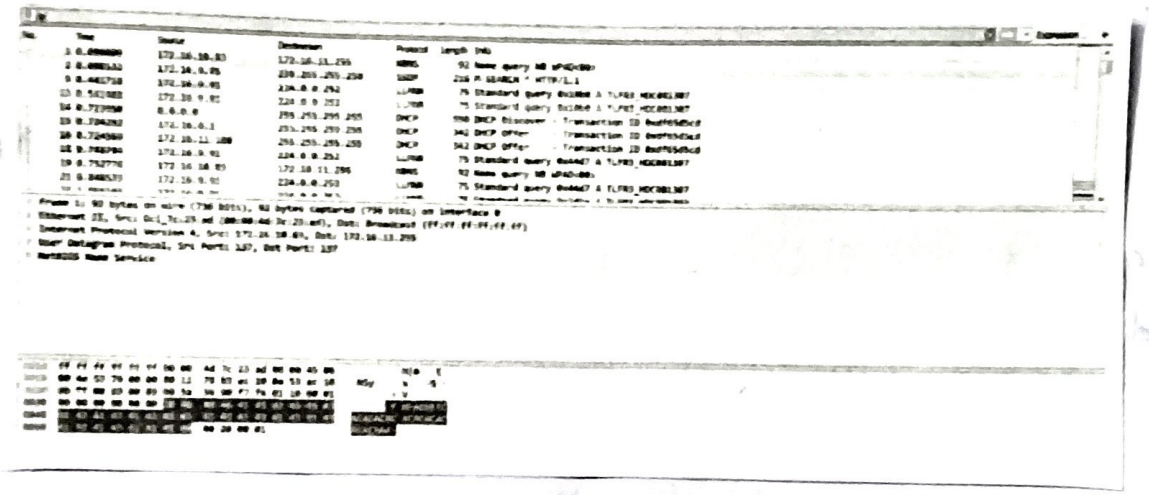
4. Create a filter to display only HTTP packets & inspect the packets

* Select Local Area Connection in wireshark
* Go to capture option
* Select stop capture automatically after 100 packets
* Then click Start capture
* Search HTTP packets ~~after~~ in search bar



5. Create a Filter to display only IP/ICMP packets & inspect the packets

* Select local area connection.
* Go to capture option
* Select ~~stop~~ capture option.
* Select stop capture automatically after 100 packets.
* Then click start capture.
* Search ICMP/IP packets in search bar



Result:

Hence the wireshark tool is studied & the network packets are captured & analysed to observe network communications.

Exp: 6 ERROR DETECTION & CORRECTION USING HAMMING CODE

AIM:

To write a program to implement error detection & correction using Hamming code concept.

Description:

Hamming code is a set of error-correction codes that can be used to detect & correct the errors that can occur when the data is transmitted from sender to reciever.

(i) Sender Program:

```
def hamming_encode(data):
    m = len(data)
    r = 0
    while 2**r < m + r + 1:
        r += 1
    code = ['0'] * (m+r)
    j = 0
    for i in range(1, len(code)+1):
        if (i & (i-1)) != 0:
            code[i-1] = data[j]
            j += 1
```

```
    for i in range(r):
        pos = 2**i
        parity = 0
        for j in range(1, len(code)+1):
            if j & pos and j != pos:
                parity ^= int(code[j-1])
        code[pos-1] = str(parity)
    return ''.join(code)

text = input("Enter the text: ")
binary = ''.join(format(ord(c), '08b') for c in text)
encoded = hamming_encode(binary)
with open("channel.txt", "w") as f:
    f.write(encoded)
print("Binary Data:", binary)
print("Hamming Encoded data:", encoded)
print("Data sent to channel.txt")
```

Reciever Program:

```
def hamming_decode(code):
    bits = list(code)
    n = len(bits)
    error_pos = 0

    parity_pos = 0
    while parity_pos <= n:
        parity = 0
        for i in range(1, n+1):
            if i & parity_pos:
                parity ^= int(bits[i-1])
        if parity:
            error_pos += parity_pos
        parity_pos *= 2
    if error_pos != 0:
        print("Error detected at position:",
              error_pos)
        bits[error_pos - 1] = '1' if
            bits[error_pos - 1] == '0'
            else '0'
        print("Error Corrected.")
    else:
        print("No Error Detected")
```

```
data = []

for i in range (1, n+1):
        if i & (i-1) != 0
                data. append (bits [i-1])

binary = "". join(data)

text = ''

for i in range (0, len(binary), 8):
        byte = binary [i : i + 8]

        if len( byte) == 8 :
                text += chr (int (byte, 2))
print ("Recieved Text : ", text)


with open ("channel. txt", "r") as f:
        recieved = f. read ( ). strip( )
hamming_decode (recieved)
```

1. Sender Program

Input:

Enter the text: A

Output

Binary data : 01000001

Hamming Encoded data : 1001 1000000 1

Data sent to channel.txt

2. Reciver Program

Output:

No error detected

Recieved text: A

Result:

Hence Hamming code program is implem
using Python.

Exp: 4 SLIDING WINDOW PROTOCOL

AIM:

To write a program to implement flow control at data link layer using Sliding Window protocol.

Program:

```
import time
import os
def sender():
    message = input("Enter the message:")
    window_size = int(input("Enter Window size:"))
    frames = list(message)
    total = len(frames)
    next_frame = 0
    while next_frame < total:
        end = min(next_frame + window_size, total)
        with open("Sender_Buffer.txt", "w") as f:
            for i in range(next_frame, end):
                f.write(f"{i+1}, {frames[i]}\n")
        print("\n Sending frames:")
```

```
for i in range (next_frame, end):
    print (f" Frame No: {i+1},
        Data : {frames [i]} ")
time. sleep (2);
reciever ()
with open ("Reciever_Buffer. txt", "r")
                                as f:
    ack =f.read (). strip ()
print ("Reciever_Buffer. txt", "r") as f :
    ack-f. read (). strip()
    print ("Recieved: ", ack )
    if ack. startswith ("ACK"):
        ack_no = int (ack. split ()[1])
        if ack_no >= end :
            next_frame = end
        else :
            print ("Resending
                Unacknowledged fra
            next_frame = max (next_frame
                ack_no)
    elif ack.startswith ("NACK"):
        nack-no= int (ack. split()[1])
        print ( f" Retransmitting from
                from frack_no}...")
        next_frame = nack_no -1
```

```
        else:
            print ("Invalid acknowledgement. Retrying...")
    print ("\n All Frames transmitted successfully!")

def reciever():
    expected_frame = 1
    if os.path.exists("Reciever_State.txt"):
        with open("Reciever_State.txt", "r") as f:
            expected_frame = int(f.read().strip())
        with open("Sender_Buffer.txt","r") as f:
            frames = f.readlines()
        ack_no = expected_frame - 1
        for line in frames:
            frame_no, data = line.strip().split(",")
            frame_no = int(frame_no)
            if frame_no == expected_frame:
                print(f"Receiver: Frame {frame_no}
                    recieved, Data: {data}")
                expected_frame += 1
                ack_no = frame_no
            else
                print(f"Reciever: Frame {frame_no}
                    rejected")
                with open("Reciver_Buffer.txt", "w")
                    as f:
                    f.write(f"NACK
                        {expected_frame}")
                return
```

```
    with open ("Reciever_State.txt", "w") as f:
        f.write (str (expected_frame))
    with open ("Reciever_Buffer.txt", "w") as f:
        f.write (f"ACK {ack no}")

if __name__ == "__main__":
    sender()
```

Sample Input:

Enter the Message : HELLO

Enter Window Size : 3

Sample Output

Sending Frames :

Frame No: 1, Data : H

Frame No: 2, Data : E

Frame No: 3, Data : L

Frame No: 4, Data : L

Reciever : Frame 1 recieved, Data : H

Reciever : Frame 2 recieved, Data : E

Reciever : Frame 3 recieved, Data : L

Result:

Hence a python program to implement Sliding Window Protocol is written & executed successfully.

Exp: 8 Nmap to Discover the hosts using Nmap Scans (ARP, TCHP, TCP/UDP) on the tryhackme Platform.

AIM:

To discover the hosts using Nmap scans (ARP, TCHP, TCP/UDP) on the Tryhackme platform.

Procedure:

Setup & Environment Installation:

* Log in to the tryhackme platform and navigate to the target room.

* Display the target unreal machine & establish a vpn connection using openVPN or launch the browser based attackbox.

ARP scan from Local Host discovery:

* Perform an ARP request scan to identify active hosts on the local network segment.

Icmp address mask Ping scan:

Send ICMP address mask request packet to identify live system by requesting subnet mask information

TCP SYN Ping Scan:

Send TCP SYN Packet directly to Port 80 (or) specified port to discover system based on SYN-ACK (or) RST responses.

cmd:

Sub map -PS-sn <target-IP/subnet>

TCP ACK Ping Scan:

Send TCP ACK packet to client RST responses from achieve host bypassing firewall security

CMD:

Sudo map -PA - Sn <target-IP*/ subnet>

UDP Ping Scan.

Send empty UDP packets to destination port to detect that via ICMP port Unreachable responses.

Cmd:

sudo nmap -PU -sn <target-IP/subnet>

Disable port scanning (port discovery only) use the -sn flag with nmap to perform host discovery exclusively without triggering subsequent port scan.

Cmd:

sudo nmap -sn <target-IP/subnet>

Result:

Thus live host discovery using nmap was successfully performed on the TryHackme platform.

Exp: 08 (b)

Configuration Of Wireless LAN Using Cisco Packet Tracer

Aim:

To design a topology with three PC's connected from Linksys Wireless routers.

Procedure:

Configure Static IP on PC & Wireless router & set SSID to MotherNetwork.

Set IP address of router to 192.168.0.1, PC0 to 192.168.0.2, PC1 to 192.168.0.3 & PC2 to 192.168.0.4.

Secure network by configuring WAP Key on router. Connect PC by using WAP Key.

Select administration tab from top menu, set username & password to admin & click on save setting.

Click on wireless tab & set default SSID to Mother Network.

Now Select wireless security & change security mode to WEP

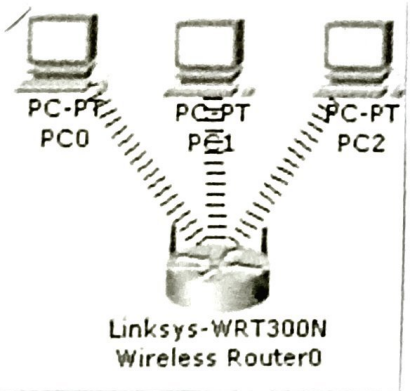
Set Key1 to 0123456789. Again go in the end of page & Click on save setting.

Double Click on PC select Desktop tab click on IP configuration, select static IP & set IP as given below.

PC	IP	Subnet Mask	Default Gateway
PC0	192.168.0.2	255.255.255.0	192.168.0.1
PC1	192.168.0.3	255.255.255.0	192.168.0.1
PC2	192.168.0.3	255.255.255.0	192.168.0.1

*Now click PC select Desktop → PC Wireless → connect tab → refresh button.

*Repeat same on PC1 & PC2.



RESULT :

A Topology with three PC's connected from Linksys wireless routers is designed using Cisco packet tracer.

Exp: 9 IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER SIMULATOR

AIM:

To implement subnetting in Cisco Packet Tracer

PROCEDURE:

classless IP subnetting is a technique that allows for more efficient use of IP addresses by allowing for subnet masks that are not just the default masks for each IP class.

* Creating a network topology:

Create a network topology in packet Tracer. Select New button in top left corner, select network & generic.

* Adding the Devices:

Select device from the bottom left corner & drag it onto the network topology Then, connect devices by dragging cable from one device's port to another device's port.

Subnetting:

To subnet the network address of 192.168.1.0/24 to provide enough space for atleast 8 addresses for end devices, we can use a /27 subnet mask. This will give us 8 subnets with 20 host addresses each.

Configuring the Devices:

Right Click on router, select CLI.

Enter:

```
# enable
#configure terminal
# interface FastEthernet0/0
# ip address {IPaddress} {Subnet Mask}
# no shutdown
# exit

interface FastEthernet0/1
ip address {IP address} {Subnet mask}

no shutdown
exit.
```

Configure the switch. Right-click on the switch & select "CLI". In the CLI, enter the following commands:

```
enable
configure terminal
interface FastEthernet0/1
switchport mode access
exit
```

To configure GigabitEthernet interface on the router,

* Right-click on the router & select CLI
* Enter the following commands:

```
enable
configure terminal
interface GigabitEthernet0/0
ip address {IP address} {subnet mask}
no shutdown
exit
```

Testing the network:

Open a command prompt on each PC & try to ping the other PC. If the ping is successful, then the network is functioning properly.

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num
●	Successful	PC4(2)	Router2	ICMP		0.000	N	12
●	Successful	PC4(2)	PC2(1)(1)	ICMP		0.000	N	13
●	Successful	PC0	Router0	ICMP		0.000	N	14

Result:

Hence Subnet is implemented using Cisco packet tracer & the connectivity is tested.

Exp. 10A INTERNETWORKING WITH ROUTERS IN CISCO PACKET TRACER SIMULATOR

AIM:

To design and configure a simple internetwork with routers in cisco packet tracer simulator.

PROCEDURE:

* Place one router & two PC's in cisco packet Tracer.

* Configure router 1 using the CLI:

```
enable
configure terminal
interface FastEthernet 0/0
ip address 192.168.10.1
255.255.255.0
no shutdown
exit
interface FastEthernet0/1
ip address 192.168.20.1
255.255.255.0
no shutdown,
```

* Connect PC0 & PC1 to the router using copper straight-through cables

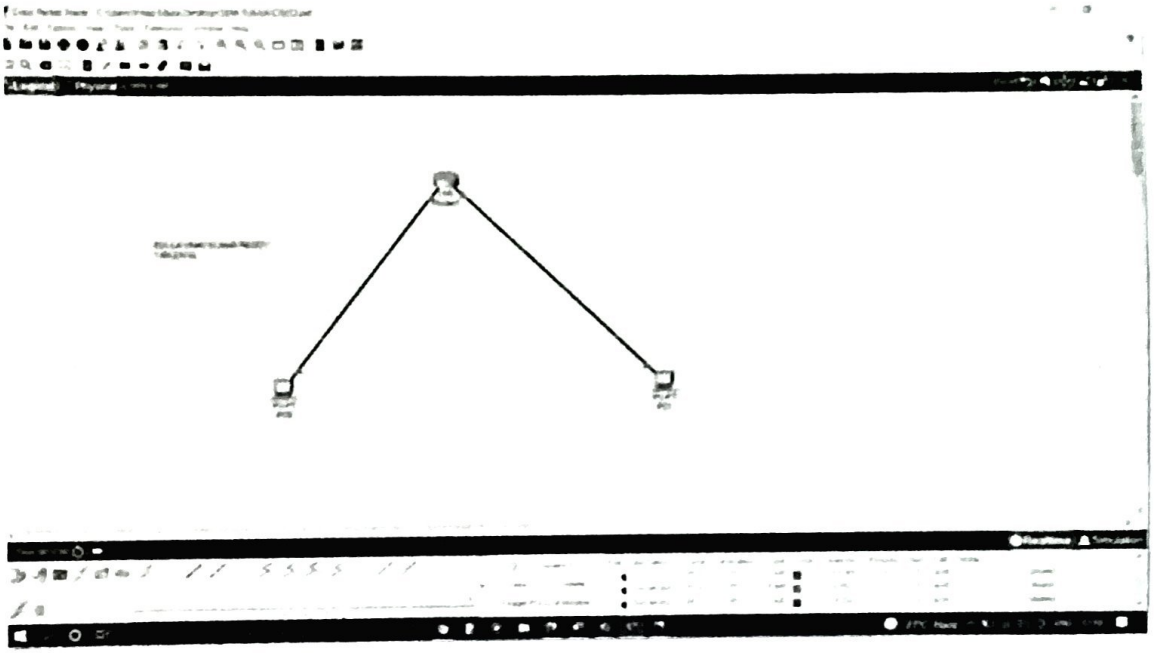
* Configure the PC's with the following IP addresses:

PC0: IP 192.168.10.2, Gateway: 192.168.10.1

PC1: IP 192.168.20.2, Gateway: 192.168.20.1

Subnet Mask for both : 255.255.255.0

* Use the add simple PDU tool to send a packet from PC0 & to PC1 & verify connectivity.



Result :
the Internetwork was configured successfully & connectivity between the two PC's was tested

Exp: 10B INTERNETWORKING USING A WIRELESS ROUTER, DHCP SERVER AND INTERNET CLOUD

AIM:

To design & configure an internetwork using a wireless router, DHCP server & Internet cloud in Cisco packet tracer & verify network connectivity.

PROCEDURE:

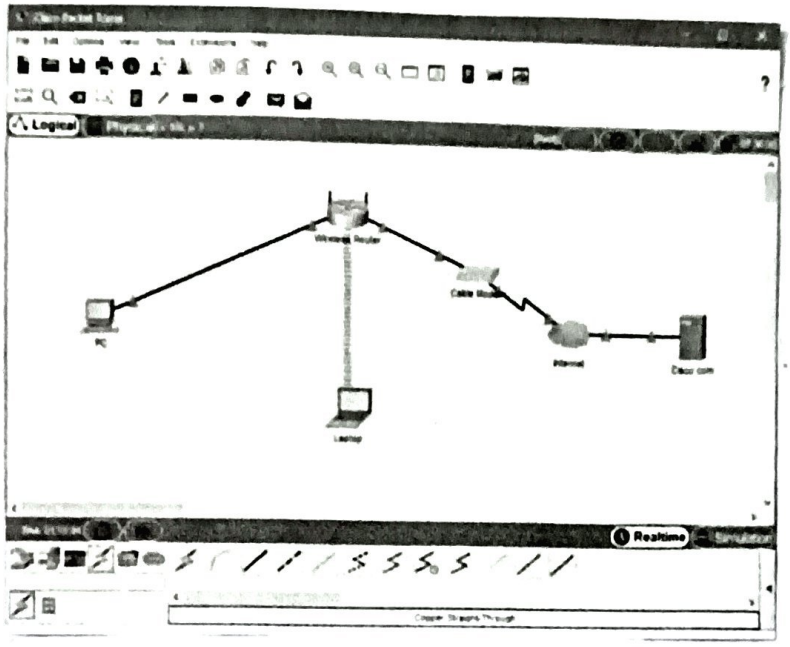
* Place a wireless router, DHCP server Internet cloud & wireless client devices in cisco packet tracer.

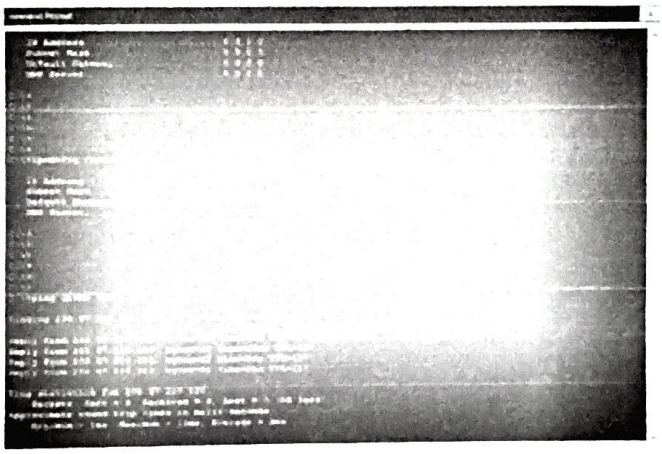
* Connect the wireless router, DHCP ~~router~~server, Internet cloud using appropriate cables.

* Configure the DHCP server with an IP address, subnet mask & default gateway and DHCP address pool.

* Connect the wireless clients to the Wifi network & enable DHCP to obtain IP address automatically.

6. Configure the router's Internet/WAN settings & verify connectivity by sending a PDU or using the ping command.





Result:

The Internetwork was designed & configured successfully using a wireless router, DHCP server & internet cloud. Wireless connectivity & automatic IP address allocation was verified

Exp 11 @A

USING CISCO PACKET TRACER

AIM:

To configure static routing between multiple networks using Cisco Packet Tracer & verify communication b/w devices connected to different networks.

PROCEDURE:

* Open Cisco packet Tracer & place two routers, two switches & two PCs in the workplace.

* Connect each PC to a switch & each switch to a router using appropriate cables. Connect the two routers using a suitable suitable interface.

* Configure IP addresses & subnet masks for the router interfaces & PCs. Assign the default gateway of each PC as the IP address of its connected router interfaces.

* Open the CLI of each router & enter privelged configuration mode.

enable
configure terminal

* Configure the router interfaces with their respective IP addresses & activate them using the no shutdown command

* Configure static routes on each router using the following command format :

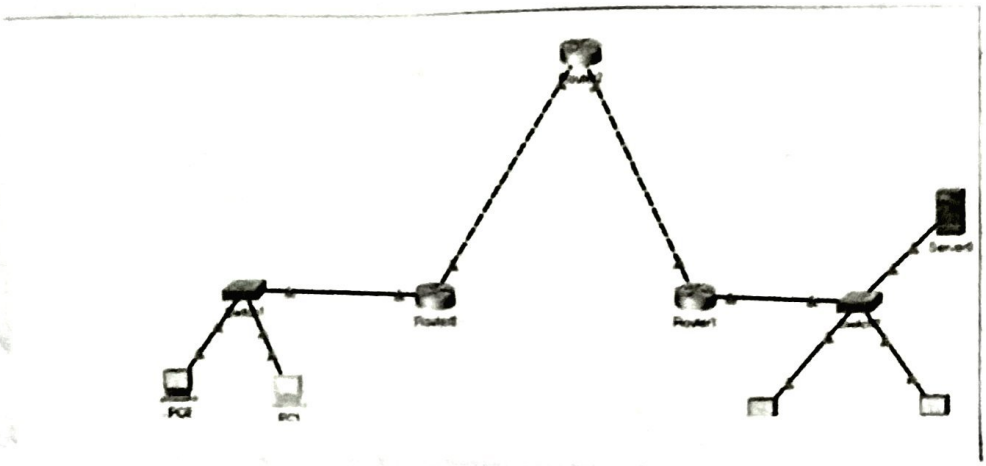
ip route <destination-network>
<subnet-mask> <next-hop-IP>

* Enter the appropriate destination network, subnet mask & next-Hop Ip address to enable communication b/w the networks.

* Verify the routing configuration using

show IP route

* Use the ping command or the add simple PDU tool to test connectivity between PC's on different networks



Result:

Static routing was successfully configured using Cisco packet Tracer & communication b/w devices in different networks was verified

Exp: 11B

SIMULATE RIP USING CISCO PACKET TRACER.

AIM:

To configure the routing information Protocol (RIP) in cisco packet tracer & verify communication between devices connected to different networks

PROCEDURE :

* Open Cisco Packet Tracer & place two routers, two switches & two PC's in the workspace.

* Connect each PC to a switch & each switch to a router. Connect the two routers using appropriate cables

* Assign IP addresses & subnet masks to all router interfaces & PC's. Configure the default gateway of each PC using its connected router's interface IP address.

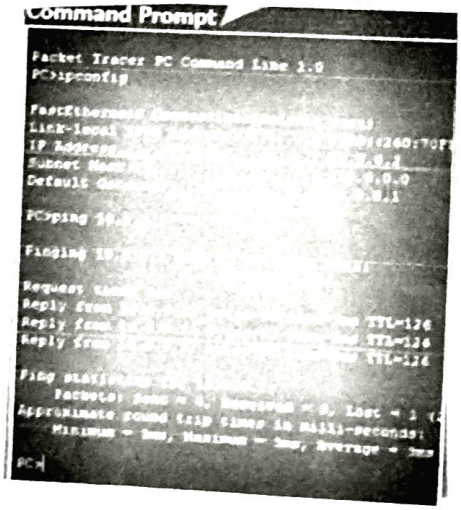
* Open the CLI of each router & enter privileged configuration mode.

* Configure RIP on router 0 using the following commands.

enable
configure terminal
router rip
version 2
no auto-summary
network 192.168.1.0
network 10.0.0.0

* Configure RIP on Router 1 similarly, Allow the routers to exchange routing information & learn routes dynamically through RIP

* Test connectivity between the PCs using the ping command



Result:

RIP was successfully configured in Cisco Packet Tracer & dynamic routing enabled communi b/w devices.

Exp: 12A

ECHO CLIENT - SERVER USING TCP/UDP SOCKETS

AIM:

To implement echo client server using TCP/UDP sockets

ALGORITHM:

* Create a socket at the server & client
* Bind the server to an IP address & port.
* Connect the TCP client
* Send a message from the client to the server.
* The Server echoes the same message back
* Display the message & close the sockets.

Code:

TCP server:

```
import socket
s = socket.socket()
s.bind(('localhost', 1234))
s.listen(1)
```

Result:

```
c, Addr = s.accept()
msg = c.recv(1024).decode()
print("Client:", msg)
c.send(msg.encode())

c.close
s.close
```

TCP Client:

```
import socket
  s=socket.socket()
s.connect(('localhost', 1234))
msg=input("Enter Message:")
s.send(msg.encode())
print("Echo:", s.recv(1024).decode())
s.close()
```

Input:
Hello Server

Output:
Client: Hello Server
Echo: Hello Server

Result:

TCP echo client-server program was implemented successfully, & the server echoed the client's message.

Exp: 12 B CHAT CLIENT-SERVER USING TCP/UDP SOCKETS

AIM:

To implement chat client-server using TCP/UDP sockets.

ALGORITHM:

* Create sockets for the client & server
* Bind the server to an IP address & port.
* Establish a connection for TCP or send datagrams for UDP.
* Read a message from the client & display it on the server.
* Send a reply from the server to the client.
* Display the recieved message & close the sockets.

Code:

```
import socket

s = socket.socket()
s.bind(('localhost', 1234))
s.listen(1)

c, addr = s.accept()
while true:
    msg = c.recv(1024).decode()
    if msg == "exit":
```

```
            break
        print ("Client:", msg)
        reply = input ("Server: ")
        c.send (reply.encode())

c.close()
s.close()
```

TCP Client:

```
import socket
s = socket.socket()
s.connect (('localhost', 1234))
while True:
    msg = input ("Client: ")
    s.send (msg.encode())
    if msg == "exit":
        break
    print("Server: ", s.recv(1024).decode())
s.close()
```

Input & Output:

Client : Hello
Server : Hi
Client : How are you?
Server : Fine

Result:

The chat client-server applications was implemented successfully using TCP sockets.

Exp: 13 IMPLEMENTING A PING PROGRAM

AIM:

To implement a ping program.

ALGORITHM:

* Import the socket module
* Get the hostname or IP address from the user
* Send a request to the destination using the ping command
* Display the response & time taken
* Display whether the destination is reachable.

Code:

```
import os
host = input ("Enter hostname or IP: ")
os.system ("ping -c 4 " + host)
```

Input:

Enter hostname or IP: google.com

Output:

64 bytes from.... time = 20 ms

64 bytes from.... time = 20 ms

64 bytes from.... time = 19 ms

64 bytes from.... time = 21 ms

4 packets transmitted, 4 received,

Result:

The Ping program was implemented successfully to check network connectivity & measure response time b/w source & dest.

Input:

Enter hostname or IP: google.com

Output:

64 bytes from.... time = 20 ms

64 bytes from.... time = 20 ms

64 bytes from.... time = 19 ms

64 bytes from.... time = 21 ms

4 packets transmitted, 4 received.

Result:

The Ping program was implemented successfully to check network connectivity & measure response time b/w source & dest.

Exp: 14 PACKET SNIFFING USING RAW SOCKETS

AIM:

To implement packet sniffing using RAW sockets.

ALGORITHM:

* Create a raw socket
* Bind the socket to the network interface.
* Capture incoming packets
* Extract and display packets details.
* Repeat the process to capture more packets.

CODE:

```
import socket

S = socket.socket (
        socket.AF_PACKET,
        socket.SOCK_RAW,
        socket.ntohs (3)
    )
```

```
while True:
    packet, addr = s.recvfrom(65535)
    print("Packet captured: ", len(packet),
                                "bytes")
```

Output:

Packet Captured : 44 bytes

Packet Captured : 60 bytes

Packet Captured : 128 bytes.

Result:

Packet-sniffing program was implemented successfully using raw sockets to capture network packets.

Exp'15 ANALYSIS OF DIFFERENT TYPES OF WEB LOGS USING WEBALIZER

AIM:

To analyze web server log files using the webalizer tools & generate statistical reports on website traffic.

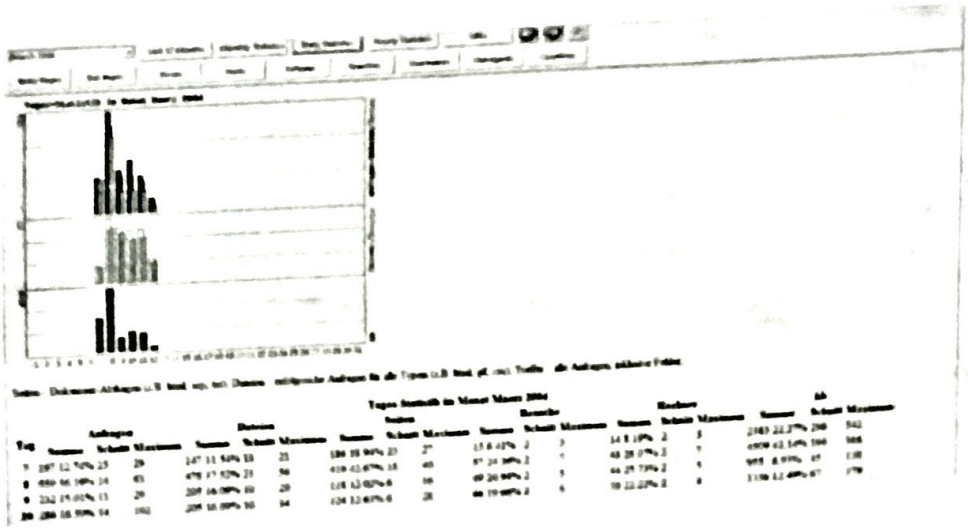
PROCEDURE:

* Install the Webalizer tool on the system.
* Provide the server access log file as input.
* Configure Webalizer to read the log file.
* Execute Webalizer to analyze the logs.
* Generate & view the HTML reports containing website statistics.

webalizer access.log.

TYPES OF WEB LOGS:

(i) Access Logs
(ii) Error Logs
(iii) Referrer Logs
(iv) User-Agent Logs.



Result:

Web server access logs were analyzed using Webalizer.